

Software Development Life Cycle

End-to-End Artifacts & Deliverables
From Business Requirements (BRD) to Release Notes and Operations

A reference guide describing each SDLC document, its purpose, owner, inputs, key contents, and how it feeds the next stage.

Author: Prabhukumar Sivamoorthy
Prabhukumarsivamoorthy@gmail.com

1. SDLC Overview

The Software Development Life Cycle (SDLC) is the structured set of phases an organization follows to plan, build, test, deploy, and maintain software. The artifacts below are presented in the order they are typically produced. In Agile they are produced iteratively per sprint/epic rather than as a single linear pass, but the logical dependency order remains the same.

Phases at a glance

Phase	Goal	Primary Artifacts
Requirements	Capture what to build & why	BRD, MRD, PRD, FRS, SRS, TRD, RTM
Design	Define how to build it	HLD, LLD, UML, API spec, DB design, security design
Implementation	Build it	Coding standards, branching strategy, code reviews
Testing	Verify & validate	Test strategy/plan, test cases, unit/integration/system/UAT
Build & Deploy	Ship it reliably	CI/CD, IaC/config mgmt, deployment & rollback plan
Release	Make it available	Release/change management, Release Notes
Operate & Maintain	Run & improve	Monitoring, incident/RCA, maintenance, retrospective

Common methodologies: **Waterfall** (sequential), **V-Model** (each build phase paired with a test phase), **Agile/Scrum** (iterative sprints), **Kanban** (continuous flow), and **DevOps** (continuous integration, delivery and feedback). The artifact set is largely the same; the cadence and formality differ.

End-to-end flow

BRD → MRD → PRD → FRS → SRS → TRD → RTM → HLD → LLD → UML diagrams → API/Interface design → Database design → Security/NFR design → Coding standards → Branching strategy → Code review → Test strategy → Test plan → Test cases → Unit testing → Integration testing → System testing → UAT → Performance testing → Security testing → Regression testing → CI/CD pipeline → Build & artifact mgmt → Config/IaC → Deployment strategy → Runbook & rollback → Release & change management → **Release Notes** → Monitoring → Incident/RCA → Maintenance → Retrospective.

2. Requirements Engineering

Defines the problem, the market need, and what the product must do — the 'why' and 'what' before any 'how'.

2.1 Business Requirements Document (BRD)

Purpose. Captures the high-level business needs, objectives, and expected value of the initiative from a stakeholder perspective.

Owner: Business Analyst / Product Sponsor | **Input:** Stakeholder interviews, business case | **Flows to:** PRD, project charter

- Business goals, objectives and success metrics (KPIs)
- Project scope, in-scope / out-of-scope boundaries
- Stakeholders, roles and high-level business processes
- Assumptions, constraints, dependencies and risks
- Cost-benefit / ROI justification and high-level timeline

2.2 Market Requirements Document (MRD)

Purpose. Describes the market opportunity, target segments and competitive landscape driving the product.

Owner: Product Marketing / Product Manager | **Input:** Market research, customer feedback | **Flows to:** PRD

- Target market, personas and customer pain points
- Competitive analysis and differentiation
- Market size, pricing and go-to-market considerations
- Prioritized market needs feeding product strategy

2.3 Product Requirements Document (PRD)

Purpose. Translates business and market needs into a concrete product definition: features, user stories and priorities.

Owner: Product Manager | **Input:** BRD, MRD | **Flows to:** FRS, SRS, design

- Product vision, goals and target users
- Features and epics with priorities (e.g., MoSCoW)
- User stories / use cases with acceptance criteria
- Non-functional expectations (usability, availability)
- Release milestones, metrics and explicit out-of-scope items

2.4 Functional Requirements Specification (FRS)

Purpose. Specifies exactly what the system must do — the functional behavior for every feature.

Owner: Business Analyst / System Analyst | **Input:** PRD | **Flows to:** SRS, HLD

- Detailed functional requirements per feature/module
- Inputs, processing rules, outputs and validations
- User roles, permissions and workflows

- Business rules and state transitions
- Error handling and edge-case behavior

2.5 Software Requirements Specification (SRS)

Purpose. The consolidated, authoritative requirements baseline combining functional and non-functional requirements (often per IEEE 830/29148).

Owner: System Analyst / Architect | **Input:** FRS, PRD | **Flows to:** HLD, test planning

- Functional requirements (from FRS)
- Non-functional requirements: performance, scalability, security, availability, compliance
- External interface requirements (UI, hardware, software, comms)
- Constraints, standards and regulatory requirements
- Acceptance criteria — the baseline tests are written against

2.6 Technical Requirements Document (TRD)

Purpose. Translates requirements into technical specifications and constraints the engineering team must satisfy.

Owner: Tech Lead / Architect | **Input:** SRS | **Flows to:** HLD, LLD

- Technology stack, platforms and target environments
- Performance, capacity and scalability targets (SLAs/SLOs)
- Integration points, protocols and data exchange formats
- Security, compliance and infrastructure requirements
- Technical constraints, assumptions and dependencies

2.7 Requirements Traceability Matrix (RTM)

Purpose. Maps each requirement forward to design, code and test cases to ensure full coverage and no scope gaps.

Owner: QA Lead / BA | **Input:** SRS/FRS | **Flows to:** Test design, audits

- Requirement ID ↔ design element ↔ test case mapping
- Coverage status and gap identification
- Bidirectional traceability for impact analysis and audits

3. Design

Defines how the system will be built — architecture, components, interfaces, data, and quality attributes.

3.1 High-Level Design (HLD) / System Architecture

Purpose. Describes the overall system architecture, major components and how they interact.

Owner: Solution Architect | **Input:** SRS, TRD | **Flows to:** LLD, infra setup

- System architecture diagram and major subsystems/services
- Component responsibilities and interactions
- Technology choices, data flow and integration overview
- Deployment topology and scalability/availability approach
- Cross-cutting concerns: security, logging, caching, resilience

3.2 Low-Level Design (LLD)

Purpose. Provides detailed internal design of each component — the blueprint developers code from.

Owner: Senior Developers / Tech Lead | **Input:** HLD | **Flows to:** Implementation

- Class/module structure, methods and responsibilities
- Detailed algorithms, data structures and pseudo-code
- Database tables, fields and relationships at field level
- API endpoint signatures, request/response schemas
- Sequence of operations, error handling and config details

3.3 UML & Design Diagrams

Purpose. Visual models that communicate structure and behavior of the system across stakeholders.

Owner: Architects / Developers | **Input:** HLD, LLD | **Flows to:** Implementation, tests

- **Use Case** — actors and system interactions (requirements)
- **Class** — static structure, attributes and relationships
- **Sequence** — time-ordered object interactions for a flow
- **Activity** — workflow / business process logic
- **State machine** — object states and transitions
- **Component & Deployment** — software units and runtime nodes
- **ER Diagram** — entities, attributes and relationships (data model)

3.4 API / Interface Design

Purpose. Defines the contracts between services and with external consumers.

Owner: Architect / Backend Lead | **Input:** HLD, LLD | **Flows to:** Implementation, integration tests

- API contracts (REST/GraphQL/gRPC) and resource models
- OpenAPI/Swagger specs, versioning and pagination strategy
- Authentication/authorization (OAuth2, JWT, API keys)

- Request/response schemas, status codes and error formats
- Rate limiting, idempotency and backward-compatibility rules

3.5 Database Design & Data Model

Purpose. Defines how data is stored, related, and accessed.

Owner: Data Architect / DBA | **Input:** LLD, ERD | **Flows to:** Implementation

- Logical and physical data models, normalization
- Schema, indexes, keys, constraints and partitioning
- Data dictionary and retention/archival policy
- Migration strategy and seed/reference data

3.6 Security, Compliance & Non-Functional Design

Purpose. Addresses quality attributes and risk so they are designed-in, not bolted-on.

Owner: Security Architect / SRE | **Input:** SRS (NFRs), TRD | **Flows to:** Implementation, security testing

- Threat model, data classification and encryption (at rest/in transit)
- AuthN/AuthZ, secrets management and audit logging
- Performance, scalability, availability and DR targets
- Compliance mapping (GDPR, HIPAA, SOC2, PCI-DSS as applicable)

4. Implementation / Development

Turning design into working, reviewed, version-controlled code.

4.1 Coding Standards & Guidelines

Purpose. Ensures consistent, readable, maintainable code across the team.

Owner: Tech Lead | **Input:** LLD, org standards | **Flows to:** Code, reviews

- Style guides, naming conventions and linting/formatting rules
- Secure coding practices and error-handling conventions
- Documentation and inline-comment expectations
- Static analysis and code-quality gates

4.2 Version Control & Branching Strategy

Purpose. Defines how code changes are organized, integrated and released.

Owner: Tech Lead / DevOps | **Input:** Coding standards | **Flows to:** CI/CD

- Branching model (GitFlow, trunk-based, GitHub Flow)
- Commit conventions and PR/merge policy
- Tagging and semantic versioning scheme
- Protected branches and required status checks

4.3 Code Review & Pull Requests

Purpose. Peer verification of correctness, design adherence, and quality before merge.

Owner: Developers / Reviewers | **Input:** Feature branch | **Flows to:** CI/CD, testing

- Review checklist: correctness, security, performance, tests
- Automated checks (lint, build, unit tests) required to pass
- Approval rules and knowledge-sharing
- Definition of Done enforcement

5. Testing & Quality Assurance

Verification (building it right) and validation (building the right thing), aligned to the V-model: unit ↔ LLD, integration ↔ HLD, system ↔ SRS, acceptance ↔ BRD/PRD.

5.1 Test Strategy

Purpose. Organization/project-level approach defining how testing is conducted overall.

Owner: QA Manager | **Input:** SRS, risk profile | **Flows to:** Test plan

- Scope, test levels and types, and quality objectives
- Tools, environments, automation vs manual split
- Entry/exit criteria, risk-based prioritization
- Roles, responsibilities and metrics

5.2 Test Plan

Purpose. Detailed plan for a specific release/feature: what, when, who and with what.

Owner: QA Lead | **Input:** Test strategy, SRS | **Flows to:** Test cases

- Features to test / not test and schedule
- Resources, environments and test data needs
- Suspension/resumption and exit criteria
- Deliverables and risk mitigation

5.3 Test Design / Test Cases

Purpose. Concrete, traceable test cases derived from requirements (linked via RTM).

Owner: QA Engineers | **Input:** Test plan, RTM | **Flows to:** Test execution

- Test case ID, preconditions, steps, expected results
- Positive, negative, boundary and edge cases
- Test data and traceability to requirements
- Techniques: equivalence partitioning, BVA, decision tables

5.4 Unit Testing

Purpose. Validates the smallest testable units (functions/classes) in isolation — owned by developers.

Owner: Developers | **Input:** LLD, code | **Flows to:** Integration testing

- White-box tests with mocks/stubs for dependencies
- High code coverage of logic and edge cases
- Runs automatically in CI on every commit/PR

5.5 Integration Testing

Purpose. Verifies that combined modules/services interact correctly across their interfaces.

Owner: Developers / QA | **Input:** HLD, API spec | **Flows to:** System testing

- Interface and data-flow verification between components
- Approaches: top-down, bottom-up, big-bang, sandwich
- Contract tests and external integration/API tests

5.6 System Testing

Purpose. Validates the complete, integrated system against the SRS in a production-like environment.

Owner: QA Team | **Input:** SRS | **Flows to:** UAT

- End-to-end functional verification of full workflows
- Non-functional checks (performance, security, usability)
- Black-box testing of the system as a whole

5.7 User Acceptance Testing (UAT)

Purpose. Business users validate the system meets real needs before go-live.

Owner: Business Users / Product | **Input:** BRD/PRD | **Flows to:** Release sign-off

- Acceptance criteria executed by end users

- Alpha/Beta testing and business-scenario validation
- Formal go/no-go sign-off for release

5.8 Specialized Testing

Purpose. Targeted quality assurance for specific quality attributes.

Owner: QA / SRE / Security | **Input:** NFRs, security design | **Flows to:** Release readiness

- **Performance/Load/Stress** — throughput, latency, scalability limits
- **Security** — SAST/DAST, pen-testing, dependency scanning
- **Regression** — confirm new changes break nothing existing (often automated)
- **Compatibility, accessibility & smoke/sanity** testing
- Defect/bug tracking and triage throughout

6. Build, Integration & Deployment (CI/CD)

Automating the path from committed code to a running, reliable release.

6.1 CI/CD Pipeline

Purpose. Automated pipeline to build, test, and deliver/deploy software continuously and repeatably.

Owner: DevOps / Platform Engineering | **Input:** Source repo, tests | **Flows to:** Deployment

- **CI:** trigger on commit → build → lint → unit/integration tests → quality gates
- **CD:** package → publish artifact → deploy to staging → automated tests → promote to prod
- Pipeline stages defined as code (e.g., GitHub Actions, GitLab CI, Jenkins, Azure DevOps)
- Automated rollback on failed health checks; approvals/gates for prod
- Security scanning (SAST/DAST/SCA) and artifact signing in-pipeline

6.2 Build & Artifact Management

Purpose. Produces versioned, immutable build artifacts and manages them.

Owner: DevOps | **Input:** CI build | **Flows to:** Deployment

- Reproducible builds and dependency management
- Versioned artifacts/containers in a registry (e.g., Artifactory, ECR)
- Provenance, SBOM and vulnerability scanning

6.3 Environment & Configuration Management (IaC)

Purpose. Provisions and configures consistent environments through code.

Owner: DevOps / SRE | **Input:** HLD, infra reqs | **Flows to:** Deployment

- Infrastructure as Code (Terraform, CloudFormation, Ansible)
- Environment parity: dev → test → staging → prod
- Secrets/config management and feature flags

6.4 Deployment Strategy

Purpose. Defines how new versions reach production with minimal risk.

Owner: DevOps / SRE | **Input:** CI/CD, runbook | **Flows to:** Release

- Blue-green, canary, rolling, or recreate strategies
- Health checks, automated promotion and traffic shifting
- Zero-downtime and database migration handling

6.5 Deployment Runbook & Rollback Plan

Purpose. Step-by-step operational guide to deploy and to recover if something fails.

Owner: DevOps / On-call | **Input:** Deployment strategy | **Flows to:** Release, ops

- Pre-deployment checklist and prerequisites
- Ordered deployment steps and validation checks
- Rollback/backout procedure and data recovery steps

- Communication and escalation plan

7. Release

Coordinating and communicating the controlled delivery of the product to users.

7.1 Release & Change Management

Purpose. Governs the controlled promotion of changes into production.

Owner: Release Manager | **Input:** All prior sign-offs | **Flows to:** Release Notes

- Release calendar, versioning and bundling of changes
- Change Advisory Board (CAB) approval and risk assessment
- Go/no-go criteria and readiness review
- Stakeholder communication plan

7.2 Release Notes

Purpose. User-facing summary of what changed in a release — the closing artifact of the delivery flow.

Owner: Product / Release Manager | **Input:** Release scope, changelog | **Flows to:** Users, support, ops

- Version number and release date
- New features and enhancements
- Bug fixes and resolved issues
- Known issues, limitations and workarounds
- Breaking changes, deprecations and upgrade/migration notes
- Security/compliance notes and contributor credits

8. Operations, Maintenance & Continuous Improvement

Post-release activities that keep the product healthy and feed learnings back into the cycle.

8.1 Monitoring & Observability

Purpose. Continuously measures system health and user impact in production.

Owner: SRE / Ops | **Input:** Release | **Flows to:** Incident mgmt, backlog

- Metrics, logs and distributed tracing (the three pillars)
- Dashboards, SLIs/SLOs/SLAs and alerting
- Synthetic checks and real-user monitoring

8.2 Incident Management & Root Cause Analysis (RCA)

Purpose. Detect, respond to, and learn from production incidents.

Owner: On-call / SRE | **Input:** Monitoring alerts | **Flows to:** Fixes, retrospective

- Severity classification and on-call escalation
- Mitigation, communication and status updates
- Blameless post-mortem and RCA (e.g., 5 Whys)

- Corrective/preventive actions tracked to closure

8.3 Maintenance & Support

Purpose. Ongoing corrective, adaptive, perfective and preventive maintenance.

Owner: Support / Dev team | **Input:** Incidents, feedback | **Flows to:** Backlog, next iteration

- Bug fixes and patches (corrective)
- Environment/dependency updates (adaptive)
- Enhancements and optimizations (perfective)
- Tech-debt reduction and hardening (preventive)

8.4 Retrospective / Continuous Improvement

Purpose. Reflects on the cycle to improve process, quality and velocity — feeding back into requirements.

Owner: Whole team | **Input:** Delivery metrics, feedback | **Flows to:** Next BRD/PRD cycle

- What went well / what to improve / action items
- Process and tooling improvements
- Updated estimates, metrics and definition of done
- Feedback loop back into the requirements phase

End of document — the cycle is iterative: insights from operations and retrospectives feed the next round of requirements.